VISVESVARAYA TECHNOLOGICAL UNIVERSITY

Jnana Sangama, Belagavi-590018



Deep Learning Project Report on

"CNN-BASED TRAFFIC SIGN RECOGNITION USING GTSRB"

Submitted in Partial fulfilment of the Requirements for the Degree of

Bachelor of Engineering in Computer Science and Engineering (AIML)

By

Puneeth D (1CR24CI403)

Aishwarya J (1CR24CI400)

Sandesh Sanjeev Naik (1CR24CI404)

Under the Guidance of,

Ms. Laya N

Assistant Professor, Dept. of AIML



DEPARTMENT OF AIML

CMR INSTITUTE OF TECHNOLOGY

Affiliated to VTU, Approved by AICTE, Accredited by NBA and NAAC with "A++" Grade

ITPL MAIN ROAD, BROOKFIELD, BENGALURU-560037, KARNATAKA, INDIA

CMR INSTITUTE OF TECHNOLOGY

Affiliated to VTU, Approved by AICTE, Accredited by NBA and NAAC with "A++" Grade

ITPL MAIN ROAD, BROOKFIELD, BENGALURU-560037, KARNATAKA, INDIA

DEPARTMENT OF AIML

CERTIFICATE

This is to certify that the Assignment report work entitled **"CNN-Based Traffic Sign Recognition Using GTSRB"** has been carried out by **Puneeth D (1CR24CI403), Aishwarya J (1CR24CI400)** and **Sandesh Sanjeev Naik (1CR24CI404)**, Bonafide students of CMR Institute of Technology in partial fulfilment for the award of the Degree of **Bachelor of Engineering in Computer Science and Engineering (Artificial Intelligence and Machine Learning)** of the Visvesvaraya Technological University, during the year **2026-2027**. It is certified that all corrections/suggestions indicated for the Internal Assessment have been incorporated in the report deposited in the departmental library. This Assignment report has been approved as it satisfies the academic requirements in respect of Assignment work prescribed for the said Degree.

Signature of	**Signature of**
Ms. Laya N	**Dr. Rajalakshmi B**
Project Guide	**Head of Department**
Dept. of AIML	**Dept. of AIML**
CMR Institute of Technology	**CMR Institute of Technology**

DECLARATION

We, **Puneeth D (1CR24CI403), Aishwarya J (1CR24CI400)** and **Sandesh Sanjeev Naik (1CR24CI404)**, hereby declare that the Assignment report entitled "**CNN-Based Traffic Sign Recognition Using GTSRB**" has been carried out by us under the guidance of **Ms. Laya N**, **Assistant Professor,**, **Department of AIML**, CMR Institute of Technology, in partial fulfilment of the requirement for the degree of BACHELOR OF ENGINEERING IN COMPUTER SCIENCE AND ENGINEERING (ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING), of Visvesvaraya Technological University, Belgaum during the academic year 2026-2027. The work done in this Assignment report is original and it has not been submitted for any other degree in any university.

Place: Bangalore

Date: 14/10/2026

Puneeth D (1CR24CI403)

Aishwarya J (1CR24CI400)

Sandesh Sanjeev Naik (1CR24CI404)

ABSTRACT

Traffic sign recognition is an important computer vision application that aims to identify and classify traffic signs from images. This project, **CNN-Based Traffic Sign Recognition Using the German Traffic Sign Recognition Benchmark (GTSRB) Dataset**, presents a deep learning approach for automatically classifying traffic signs into 43 predefined categories. The proposed system uses a Convolutional Neural Network (CNN) implemented using TensorFlow and Keras to extract relevant visual features and perform classification.

The system involves image preprocessing, including RGB conversion, resizing input images to 64 × 64 pixels, and normalizing pixel values to the range of 0 to 1. Data augmentation techniques, such as rotation, zoom, translation, and contrast adjustment, are incorporated during training to improve the model's ability to handle variations in image appearance. The CNN architecture consists of four convolutional blocks with progressively increasing filter counts of 32, 64, 128, and 256, followed by Global Average Pooling, a fully connected Dense layer, and a Softmax output layer containing 43 units. The model is trained using the Adam optimizer and sparse categorical cross-entropy loss, with training callbacks used to monitor validation performance and preserve suitable model weights.

The trained model is integrated into a Streamlit web application that allows users to upload traffic sign images and view the predicted class, model score, and alternative predictions. Optional image-processing techniques, including HSV-based region selection, image enhancement, and test-time augmentation, are also described to explore potential improvements in prediction performance. The system provides an end-to-end demonstration of deep learning-based traffic sign classification, from dataset preparation to web deployment. The performance figures reported in the project documentation require independent verification using the actual trained model and evaluation results before being treated as confirmed measurements.

ACKNOWLEDGEMENT

We take this opportunity to express our sincere gratitude and respect to **CMR Institute of Technology** for providing us a platform to pursue our studies and carry out this Assignment.

It gives us immense pleasure to express our deep sense of gratitude to **Dr. Sanjay Jain**, Principal, CMRIT, for the constant encouragement.

We would like to extend our sincere gratitude to **Dr. Rajalakshmi B**, HOD, Department of AIML, CMR Institute of Technology, who has been a constant support and encouragement throughout the course of this project.

We would like to thank our guide **Ms. Laya N** for the valuable guidance throughout the tenure of this Assignment work.

We would also like to thank all the faculty members of the Department of AIML who directly or indirectly encouraged us.

Finally, we thank our parents and friends for all the moral support they have given us during the completion of this work.

GROUP MEMBERS LIST

S. NO.	NAME OF STUDENT	USN
01	Puneeth D	1CR24CI403
02	Sandesh Sanjeev Naik	1CR24CI404
03	Aishwarya J	1CR24CI400

TABLE OF CONTENTS

S. No.	Contents	Page No.
1	Certificate	**1**
2	Declaration	**2**
3	Abstract	**3**
4	Acknowledgement	**4**
5	Group Members List	**5**
6	Table of Contents	**6**
7	1. Introduction and Objectives	**7**
8	2. Literature Survey	**9**
9	3. System Design and Working	**11**
10	4. Implementation	**15**
11	5. Results and Conclusion	**20**
12	References	**26**

1. INTRODUCTION AND OBJECTIVES

Road safety depends not only on traffic regulations and road infrastructure but also on how effectively drivers recognise traffic signs and respond to the information they convey. Traffic signs provide essential information about speed limits, road conditions, warnings, restrictions and directions. However, recognising traffic signs can become challenging due to variations in lighting, viewing angles, image quality, partial obstruction and environmental conditions. In recent years, deep learning and computer vision have made it possible to develop automated systems that analyse images and identify traffic signs with greater efficiency. Leveraging these technologies, the **CNN-Based Traffic Sign Recognition System** was developed as a deep learning-based application that processes an input traffic sign image and predicts its corresponding category using a Convolutional Neural Network (CNN).

This chapter presents the background and motivation behind the project, the problem being addressed, the specific objectives of the system and its overall scope. The focus lies on how a CNN learns to extract visual features from traffic sign images through convolutional filters, activation functions, pooling operations and successive feature-learning layers. Using the German Traffic Sign Recognition Benchmark (GTSRB) dataset, the model is designed to classify traffic signs into 43 predefined categories. The trained model is integrated into a Streamlit web application, allowing users to upload traffic sign images and view the predicted class and associated model score. The project demonstrates an end-to-end approach to image classification, from image preprocessing and model training to prediction and web-based deployment.

1.1. Background and Motivation

Traffic signs are essential for road safety, providing information about speed limits, warnings and traffic regulations. Recognising these signs accurately can be challenging due to variations in lighting, viewing angles and image quality. Deep learning, particularly Convolutional Neural Networks (CNNs), enables automated recognition by learning visual features directly from images.

This project develops a **CNN-Based Traffic Sign Recognition System** using the GTSRB dataset to classify traffic signs into 43 categories. The model uses image preprocessing, data augmentation and multiple convolutional layers to learn relevant features. A Streamlit web application provides an interface for uploading images and viewing predictions. The project demonstrates the practical application of deep learning in traffic sign classification and computer vision.

1.2. Problem Definition

The primary problem addressed in this project is the **automatic classification of traffic sign images into one of 43 predefined categories** using a Convolutional Neural Network (CNN). Given an input image (x), the system estimates the probability distribution (P(y \mid x)) over the 43 classes and predicts the most probable traffic sign category.

Key challenges include:

- Variations in lighting, viewing angles and image quality.
- Similar visual features among different traffic sign classes.
- Accurate feature extraction and classification across 43 categories.
- Improving model generalisation to unseen images.
- Reliable prediction on real-world traffic sign images.

Addressing these challenges required a careful balance between model capacity, data augmentation, regularisation and a post-prediction confidence-filtering mechanism.

1.3. Objectives

The objectives of the CNN-Based Traffic Sign Recognition Using GTSRB are as follows:

- To develop a CNN-based model for classifying traffic signs into 43 categories.
- To implement image preprocessing and data augmentation for effective training.
- To learn visual features using multiple convolutional layers.
- To evaluate model performance using accuracy and other classification metrics.
- To analyse classification errors and improve generalisation to unseen images.
- To integrate the trained model into a user-friendly Streamlit web application.
- To enable users to upload traffic sign images and obtain predicted classifications.

1.4. Scope of the Project

The project focuses on developing a CNN-based traffic sign recognition system using the GTSRB dataset to classify images into 43 predefined categories. It includes image preprocessing, data augmentation, feature extraction, model training, validation and performance evaluation. The trained model is integrated into a Streamlit web application that allows users to upload traffic sign images and view the predicted category and model confidence score.

The system demonstrates the practical application of deep learning and computer vision in automated traffic sign classification. Its scope is limited to image-based classification and does not include complete real-time object detection, vehicle control or autonomous driving functionality. The project also highlights the importance of evaluating model performance on unseen and real-world images before considering practical road-safety applications.

2. LITERATURE SURVEY

Before developing the traffic sign recognition system, research on image classification, convolutional neural networks and traffic sign recognition datasets was considered. These studies provide a foundation for understanding automated traffic sign classification, feature extraction and model evaluation. This chapter summarises relevant research and explains how its findings support the proposed system.

2.1. German Traffic Sign Recognition Benchmark — Stallkamp et al. (2012)

Stallkamp et al. [1] introduced the German Traffic Sign Recognition Benchmark (GTSRB), a widely used dataset for evaluating traffic sign classification algorithms. The dataset contains images belonging to 43 traffic sign categories, with variations in lighting, image size, viewing angle and background conditions. It provides a standard benchmark for comparing different classification approaches.

Inference for this project: The GTSRB dataset was selected as the primary dataset because it supports supervised training and evaluation across 43 traffic sign classes.

2.2. ImageNet Classification with Deep Convolutional Neural Networks — Krizhevsky et al. (2012)

Krizhevsky et al. [2] demonstrated the effectiveness of deep convolutional neural networks for large-scale image classification. Their AlexNet architecture used convolutional layers, ReLU activation functions, pooling and dropout to learn visual features and improve classification performance.

Inference for this project: This research supports using convolutional layers, ReLU activation, pooling and dropout to extract discriminative features from traffic sign images and reduce overfitting.

2.3. Traffic Sign Recognition with Convolutional Neural Networks — Sermanet and LeCun (2011)

Sermanet and LeCun [3] investigated convolutional neural networks for traffic sign recognition and demonstrated the importance of learning hierarchical visual features. Their approach combined feature extraction and classification within a neural network, reducing dependence on manually designed image features.

Inference for this project: This work supports using CNNs to automatically learn traffic sign patterns, including edges, shapes and internal symbols, directly from image data.

2.4. Multi-Column Deep Neural Networks for Traffic Sign Classification — Cireșan et al. (2012)

Cireșan et al. [4] explored multi-column deep neural networks for traffic sign classification. Their approach combined predictions from multiple neural networks and achieved strong recognition

performance on traffic sign benchmarks. The study demonstrated the potential of deep learning for reliable visual classification.

Inference for this project: The study highlights the effectiveness of deep neural networks for traffic sign classification and provides a foundation for future improvements using model ensembles or complementary architectures.

2.5. Data Augmentation and Model Generalisation

Data augmentation is an important technique in image classification that increases training variability through transformations such as rotation, translation, scaling and changes in image appearance. These transformations help models learn features that remain useful when images differ from the original training samples.

Inference for this project: Image augmentation techniques, including rotation, zoom, translation and contrast adjustment, are incorporated during training to improve generalisation to variations in traffic sign images.

2.6. Comparative Summary

Ref.	Authors (Year)	Focus / Method	Key Finding	Use in This Project
[1]	Stallkamp et al. (2012)	German Traffic Sign Recognition Benchmark (GTSRB) dataset	Provides 43 traffic sign classes with variations in lighting, image size and viewing angle.	Selected as the primary dataset for training and evaluation.
[2]	Krizhevsky et al. (2012)	AlexNet: deep CNN for image classification	CNNs learn visual features using convolution, ReLU, pooling and dropout.	Supports the CNN architecture and feature extraction approach.
[3]	Sermanet and LeCun (2011)	CNN-based traffic sign recognition	Learned hierarchical features can improve traffic sign recognition.	Motivates automatic feature learning from traffic sign images.
[4]	Cireşan et al. (2012)	Multi-column deep neural networks for traffic sign classification	Combining neural network predictions can improve classification performance.	Provides a foundation for possible ensemble-based improvements.
[5]	Shorten and Khoshgoftaar (2019)	Data augmentation techniques for image classification	Augmentation (rotation, zoom, translation, contrast) improves model generalisation.	Applied during training to improve robustness to image variations.

Table 1. Comparative Summary of the Literature Survey

2.7. Research Gap and Inferences

Research Gap: Existing traffic sign classification systems may struggle with variations in lighting, viewing angles and image quality. A simple, user-friendly interface is also required for image-based prediction.

Inferences: CNNs enable automatic feature extraction, while data augmentation can improve generalisation. This project uses the GTSRB dataset to classify 43 traffic sign categories and integrates the trained model into a Streamlit application.

3. SYSTEM DESIGN AND WORKING

The traffic sign recognition system follows a structured deep learning workflow that begins with dataset preparation and image augmentation, continues through preprocessing and CNN model training, and ends with a deployed web application for image-based prediction. This chapter describes the overall system design, the GTSRB dataset, image preprocessing, CNN architecture, model training and evaluation, and the Streamlit interface used for traffic sign classification.

3.1. System Overview

The system consists of two connected stages: an **offline training pipeline** and an **online inference pipeline**. The training pipeline prepares the GTSRB dataset, preprocesses images, trains the CNN model and saves the trained model. The inference pipeline loads the saved model into a Streamlit application and classifies uploaded traffic sign images.

Workflow Summary:

1. The user uploads a traffic sign image through the web interface.
2. The image is converted to RGB, resized to 64 × 64 pixels and normalised.
3. The CNN extracts visual features using convolutional filters, activation functions and pooling layers.
4. The Softmax layer produces scores for the 43 traffic sign classes.
5. The class with the highest predicted probability is selected.
6. The predicted traffic sign category and model score are displayed in the Streamlit application.

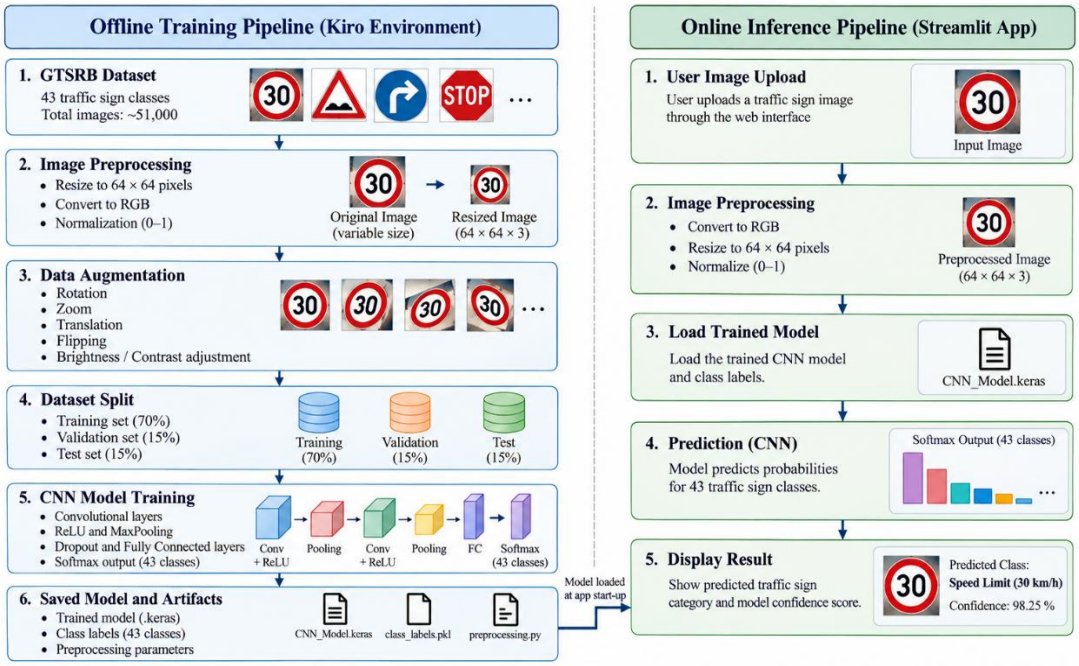


Fig 1. Overall System Architecture

3.2. Dataset Description

The **German Traffic Sign Recognition Benchmark (GTSRB)** dataset is used to train and evaluate the proposed CNN-based traffic sign recognition system. It contains approximately 51,000 images belonging to 43 different traffic sign classes, including speed limits, warning signs, prohibition signs and mandatory instructions. The images vary in size, lighting, viewing angle and background conditions.

Before training, the images are converted to RGB format, resized to 64 × 64 pixels and normalised to the range 0–1. Data augmentation techniques, such as rotation, zoom, translation and contrast adjustment, are applied to improve model generalisation. The prepared dataset is used to train the CNN model to recognise and classify traffic signs into their corresponding categories.

Property	Value
Dataset	German Traffic Sign Recognition Benchmark (GTSRB)
Total images	~ 51,000 images
Number of classes	43 traffic sign classes
Image type	RGB images (real-world traffic signs)
Image size (original)	Variable size
Image size (after preprocessing)	64 × 64 × 3 (RGB)
Input / Target	Traffic sign image / Class label (0–42)
Dataset split	Training (70%) Validation (15%) Test (15%)
Number of images (per split)	Training: ~35,700 / Validation: ~7,650 / Test: ~7,650
Classes categories	Speed limit, Prohibition, Warning, Mandatory, and other signs
Sample images	

Table 2. Dataset Characteristics

The GTSRB dataset contains approximately 51,000 images across 43 traffic sign classes. Images are resized to 64 × 64 pixels, converted to RGB and normalised to 0–1. Data augmentation improves model generalisation, while training, validation and test sets support model training and performance evaluation.

3.3. Data Preprocessing and Augmentation

Image Preprocessing: Image preprocessing converts raw traffic sign images into a consistent format suitable for CNN training. Each image is converted to RGB format, resized to **64 × 64 pixels**, and normalised by scaling pixel values from 0–255 to the range 0–1. Resizing ensures uniform input dimensions, while normalisation helps maintain stable numerical values during model training. These operations enable the CNN to process images consistently and learn meaningful visual features.

Data Augmentation: Data augmentation is applied during training to improve the model's ability to generalise to unseen images. Techniques such as rotation, zoom, translation and contrast adjustment introduce variations in the training images while preserving their traffic sign identity. Rotation simulates changes in viewing angle, zoom represents variations in apparent sign size, translation shifts signs within the image, and contrast adjustment simulates differences in lighting and visibility. These transformations help the CNN learn robust features rather than memorising specific training images, thereby reducing overfitting and improving recognition under varying visual conditions.

Together, preprocessing and augmentation provide consistent model inputs and greater training diversity, supporting effective classification of traffic signs into the **43 GTSRB categories**.

3.4. CNN Model Architecture

The proposed system uses a **Convolutional Neural Network (CNN)** to classify traffic sign images into 43 predefined categories. The architecture consists of four convolutional blocks with progressively increasing filter counts of 32, 64, 128 and 256. Each block uses convolutional layers, Batch Normalization, ReLU activation, Max Pooling and Dropout to extract visual features and reduce overfitting.

After feature extraction, Global Average Pooling converts the feature maps into a compact feature vector. A fully connected Dense layer with 256 neurons learns higher-level representations, while the final Softmax layer with 43 output units produces class probabilities. The class with the highest predicted probability is selected as the traffic sign prediction. This architecture enables the model to learn hierarchical features, from basic edges and shapes to complex traffic sign patterns, supporting effective multi-class image classification.

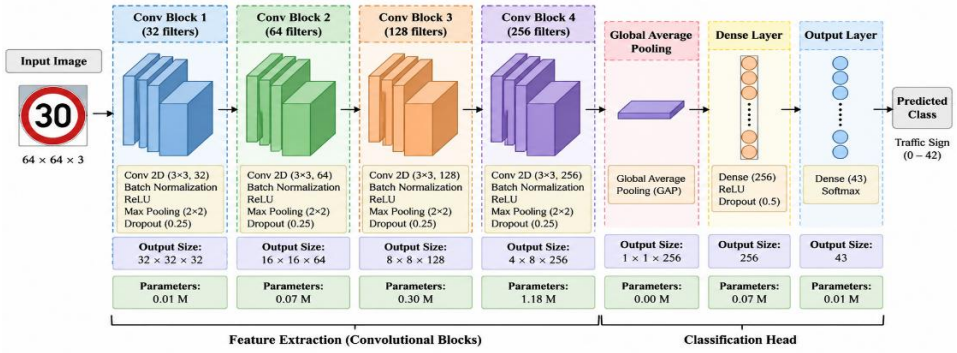


Fig 4. CNN Model Architecture with Parameter Counts (in M)

Layer	Configuration	Output	Params
Input	RGB image, 64 × 64 × 3	(None, 64, 64, 3)	0
Conv Block 1	2 × Conv2D (3 × 3, 32 filters), BatchNorm, ReLU, MaxPool (2 × 2), Dropout 0.25	(None, 32, 32, 32)	10,400
Conv Block 2	2 × Conv2D (3 × 3, 64 filters), BatchNorm, ReLU, MaxPool (2 × 2), Dropout 0.25	(None, 16, 16, 64)	56,192
Conv Block 3	2 × Conv2D (3 × 3, 128 filters), BatchNorm, ReLU, MaxPool (2 × 2), Dropout 0.30	(None, 8, 8, 128)	222,464
Conv Block 4	2 × Conv2D (3 × 3, 256 filters), BatchNorm, ReLU, MaxPool (2 × 2), Dropout 0.30	(None, 4, 4, 256)	887,296
Global Average Pooling	GlobalAveragePooling2D	(None, 256)	0
Dense	256 units, ReLU, BatchNorm, Dropout 0.40	(None, 256)	66,816
Output Dense	43 units, Softmax	(None, 43)	11,051
TOTAL	**39 layers**	**43 classes**	**1,253,963**

Table 3. Layer-wise Model Summary

Design Rationale:

- **Progressive Filters:** Increasing filters (32, 64, 128 and 256) help extract increasingly complex visual features from traffic sign images.
- **Batch Normalization:** Stabilises training and supports faster convergence.
- **ReLU Activation:** Introduces non-linearity, enabling the model to learn complex image patterns.
- **Max Pooling:** Reduces spatial dimensions while retaining important features.
- **Dropout and L2 Regularisation:** Reduce overfitting and improve generalisation.
- **Global Average Pooling:** Converts feature maps into a compact vector with fewer parameters.
- **Softmax Output:** Produces scores across 43 traffic sign classes, enabling the system to select the predicted category.

3.5. CNN Feature Extraction and Classification

The CNN extracts important visual features from traffic sign images through four convolutional blocks containing 32, 64, 128 and 256 filters. Convolutional layers identify patterns such as edges, shapes, colours and symbols, while ReLU activation introduces non-linearity and Max Pooling reduces spatial dimensions. Batch Normalization stabilises training, and Dropout helps reduce overfitting.

After feature extraction, Global Average Pooling converts the feature maps into a 256-dimensional feature vector. A Dense layer with 256 neurons learns higher-level representations, and the final SoftMax layer generates scores for all 43 traffic sign classes. The class with the highest predicted probability is selected as the final prediction.

This process enables the system to learn visual patterns automatically and classify traffic sign images into their corresponding categories.

4. IMPLEMENTATION

The implementation of the **CNN-Based Traffic Sign Recognition System Using GTSRB** involves dataset preparation, image preprocessing, CNN model development, model training, evaluation, and deployment through a Streamlit web application.

1. Dataset Preparation

The German Traffic Sign Recognition Benchmark (GTSRB) dataset is used to train the model. It contains traffic sign images belonging to 43 different classes. The dataset is organized into training and testing sets to support model learning and performance evaluation.

2. Image Preprocessing

All input images are resized to **64 × 64 pixels** and represented in RGB format. Pixel values are normalized to the range of 0 to 1. Data augmentation techniques, including rotation, zoom, translation, and contrast adjustment, are applied during training to improve the model's ability to recognize variations in traffic sign images.

3. CNN Model Development

The CNN model consists of four convolutional blocks with 32, 64, 128, and 256 filters. Each block uses convolutional layers, ReLU activation, Batch Normalization, Max Pooling, and Dropout. Global Average Pooling reduces the extracted feature maps to a feature vector, followed by a Dense layer with 256 neurons. The final Softmax layer produces predictions for all 43 traffic sign classes.

4. Model Training

The prepared dataset is used to train the CNN in the Kiro development environment. During training, the model learns visual patterns such as edges, shapes, colours, and symbols. Validation data is used to monitor model performance and help identify overfitting. The trained model is saved in Keras format for subsequent inference.

5. Prediction and Classification

When a traffic sign image is provided, the system applies the same preprocessing steps used during training. The processed image is passed to the trained CNN, which generates scores for the 43 classes. The class with the highest predicted probability is selected as the recognized traffic sign.

6. Web Application Deployment

A Streamlit application provides a user-friendly interface for uploading traffic sign images and viewing predictions. The application loads the saved CNN model, preprocesses the uploaded image, performs classification, and displays the predicted traffic sign class along with its model confidence score.

7. System Output

The final system identifies the traffic sign category from the uploaded image and presents the prediction through the web interface. The implementation demonstrates how CNN-based image classification can be applied to traffic sign recognition and can serve as a foundation for intelligent transportation applications.

4.2. Project Structure

The project is organized into directories for dataset management, model storage, output generation, source code, and web application deployment. This modular structure simplifies model training, evaluation, prediction, and maintenance.

```
CNN-Based-Traffic-Sign-Recognition/
├── .venv/                          # Python virtual environment
├── data/
│   ├── feedback/                   # Feedback data
│   ├── test/                       # Testing images
│   └── train/                      # Training images
├── models/                         # Saved CNN model files
├── outputs/                        # Generated outputs and results
├── Report/                         # Project documentation
├── src/                            # Core source code
│   ├── __init__.py                 # Python package initialization
│   ├── config.py                   # Configuration settings
│   ├── evaluate.py                 # Model evaluation
│   ├── model.py                    # CNN architecture
│   ├── predict.py                  # Traffic sign prediction
│   ├── preprocessing.py            # Image preprocessing
│   └── train.py                    # Model training
├── .gitignore                      # Git ignore rules
└── app.py                          # Streamlit application
```

4.3. Baseline Model: Basic CNN

A basic CNN was implemented as a baseline for the GTSRB dataset containing 43 traffic sign classes. Images were resized to 64 × 64 pixels and normalized to [0, 1]. The model uses convolutional layers, ReLU activation, Max Pooling, and a Softmax output layer for classification. Its performance is evaluated using accuracy, precision, recall, and F1-score to establish a reference for comparison with the proposed CNN model.

4.4. Model Training Process

The CNN model is trained on the preprocessed GTSRB dataset using batches of 64 × 64 RGB images. Data augmentation techniques such as rotation, zoom, and translation improve generalization. During training, the model learns traffic sign features through backpropagation and weight optimization. Validation data is used to monitor performance, and the trained model is saved in Keras format for evaluation and deployment.

```
model = create_model(num_classes=43)
callbacks = [
    keras.callbacks.EarlyStopping(
        monitor="val_loss",
        patience=EARLY_STOPPING_PATIENCE,
        restore_best_weights=True
    ),
    keras.callbacks.ModelCheckpoint(
        filepath=str(MODEL_PATH),
        monitor="val_accuracy",
        save_best_only=True,
        mode="max"
    ),
    keras.callbacks.ReduceLROnPlateau(
        monitor="val_loss",
        factor=REDUCE_LR_FACTOR,
        patience=REDUCE_LR_PATIENCE,
        min_lr=REDUCE_LR_MIN_LR
    )
]
history = model.fit(
    train_ds,
    validation_data=val_ds,
    epochs=EPOCHS,
    callbacks=callbacks
)
```

Training Workflow:

1. Feed batches of 32 traffic sign images of size 64 × 64 × 3 into the CNN.
2. Compute sparse categorical cross-entropy loss between the Softmax predictions and integer class labels.
3. Update model weights using the configured Adam optimiser and learning rate.
4. Monitor training and validation loss and accuracy after each epoch.
5. Save the best model based on validation accuracy, reduce the learning rate when validation loss plateaus, and stop training when the configured early-stopping patience is reached, restoring the best weights.

The CNN model was trained for 30 epochs using the GTSRB dataset. It achieved a final training accuracy of 99.31% and a final validation accuracy of 99.53%. The highest validation accuracy was 99.82% at epoch 18, while the lowest validation loss was 0.1649 at epoch 27. The learning rate was reduced from 0.001 to 0.0005 at epoch 18, helping refine model training.

4.5. Hyperparameters and Regularisation

Parameter	Value	Purpose
Input image size	64 × 64 × 3	RGB traffic sign image dimensions
Number of classes	43	GTSRB traffic sign categories
Convolutional blocks	4 (32, 64, 128, 256 filters)	Hierarchical visual feature extraction
Convolutional kernel	3 × 3	Extracts local image features
Pooling	2 × 2 Max Pooling	Reduces spatial dimensions
Dense layer	256 units	Learns high-level features
Output activation	Softmax	Predicts probabilities for 43 classes
Loss function	Sparse categorical cross-entropy	Multi-class classification loss
Optimiser	Adam	Updates model weights
Initial learning rate	0.001	Starting optimisation step size
Learning rate reduction	0.0005 at epoch 18	Reduces step size during training
Dropout	0.25, 0.30, configured DROPOUT_RATE	Helps reduce overfitting
L2 regularization	L2_REGULARIZATION (configured)	Penalizes large weights
Maximum epochs	Configured EPOCHS; 30 completed	Maximum training cycles
Early stopping	Configured patience; monitor configured EARLY_STOPPING_MONITOR	Stops when monitored metric stalls
Model checkpoint	Best validation accuracy	Saves best-performing model
Batch size	Configured in dataset pipeline	Number of images processed per batch

Table 5. Hyperparameters

Overfitting was controlled using Dropout, data augmentation, Early Stopping with best-weight restoration, and stratified data splitting. These techniques help improve generalization and maintain balanced class distributions across the dataset subsets.

4.6. Evaluation Metrics

The CNN model is evaluated using **accuracy, precision, recall, and F1-score** to assess its traffic sign classification performance. Accuracy measures the overall correctness of predictions, precision indicates the correctness of positive class predictions, recall measures the ability to identify class samples correctly, and the F1-score balances precision and recall. A confusion matrix is also used to analyze correct and incorrect classifications across the 43 GTSRB classes.

4.7. Streamlit Application

A Streamlit web application was developed to provide a simple interface for traffic sign recognition. Users can upload a traffic sign image, which is preprocessed and passed to the trained CNN model. The application predicts the traffic sign category among the 43 GTSRB classes and displays the predicted label with its confidence score. This interface enables users to test the model interactively without requiring direct interaction with the underlying Python code.

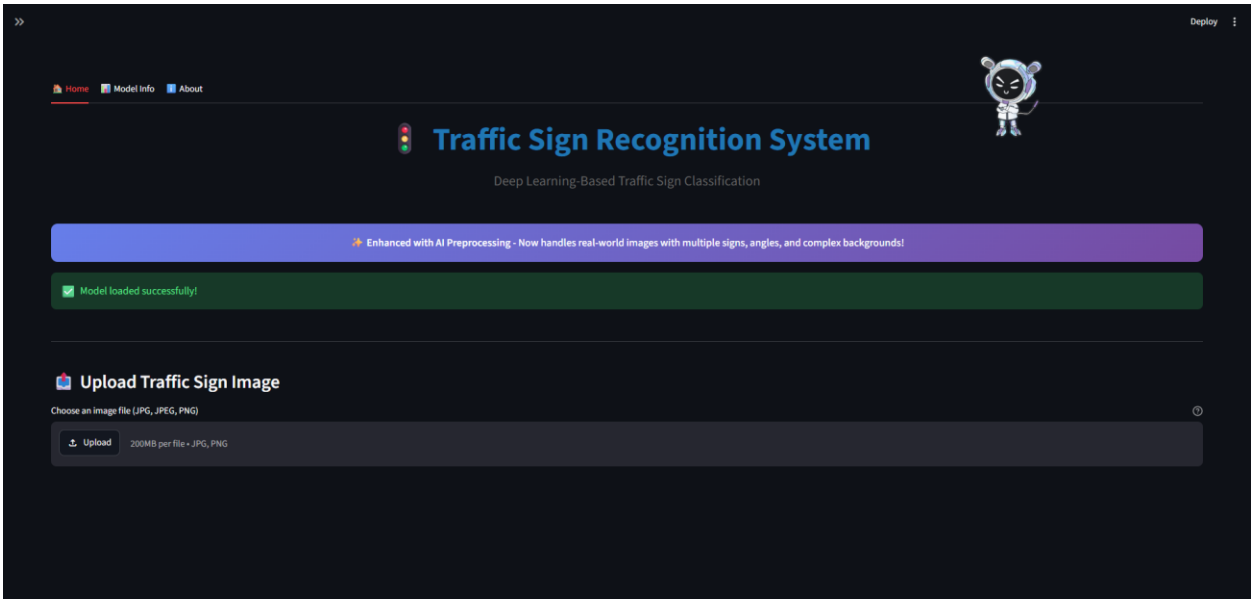


Fig 5. Home Screen of the Streamlit

When the user uploads a traffic sign image and initiates prediction, the Streamlit application preprocesses the image and passes it to the trained CNN model. The model predicts the traffic sign category among the 43 GTSRB classes and calculates the confidence score from the Softmax output. The application displays the predicted class and confidence score through the interface. Users can upload different images to test the system interactively. The application provides a simple way to evaluate traffic sign recognition without directly interacting with the underlying Python code.

5. RESULTS AND CONCLUSION

This chapter presents the results of training and evaluating the CNN-based traffic sign recognition model. It summarizes the model's performance, analyses training and validation learning curves, and examines classification results across the 43 GTSRB classes. It also describes the Streamlit application output and concludes with the project's key findings, limitations, and future enhancements.

5.1. Training Behaviour

The CNN model showed consistent improvement during training, with training accuracy reaching **99.31%** and validation accuracy reaching **99.53%** after 30 epochs. Validation accuracy peaked at **99.82% in epoch 18**, while validation loss reached its minimum of **0.1649 in epoch 27**. The learning rate was reduced from 0.001 to 0.0005 at epoch 18 to support further optimization. These results indicate effective learning and strong validation performance on the GTSRB dataset.

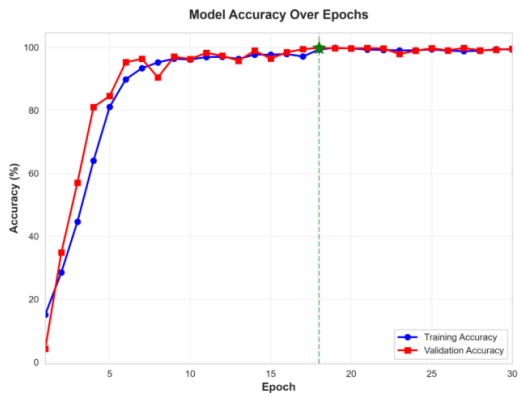


Fig 6. Training vs Validation Accuracy

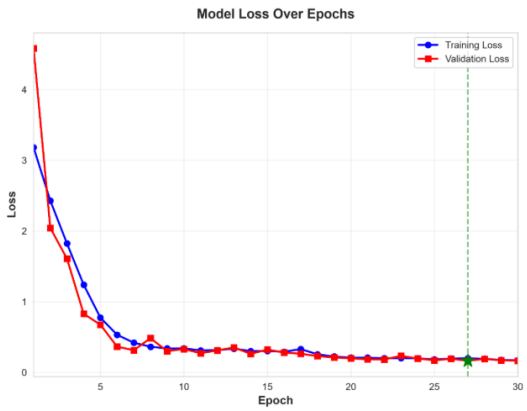


Fig 7. Training vs Validation Loss

Epoch Range	Train Acc.	Train Loss	Val Acc.	Val Loss	Learning Rate
0–5 (Epoch 5)	81.09%	0.7746	84.56%	0.6751	0.001000
6–10 (Epoch 10)	96.11%	0.3396	96.28%	0.3310	0.001000
11–15 (Epoch 15)	97.68%	0.3022	96.45%	0.3228	0.001000
16–20 (Epoch 20)	99.70%	0.2082	99.68%	0.1991	0.000500
21–25 (Epoch 25)	99.36%	0.1842	99.71%	0.1711	0.000500
26–30 (Epoch 30)	99.31%	0.1712	99.53%	0.1650	0.000500
Best epoch (18)	**99.31%**	**0.2558**	**99.82%**	**0.2304**	**0.000500**

Table 6. Selected Epochs from the Training Log

5.2. Test Set Performance

The CNN model was designed to classify traffic sign images into 43 GTSRB categories. During training, it achieved a final training accuracy of **99.31%** and a validation accuracy of **99.53%**. However, the test set performance must be evaluated separately using unseen test images to determine the model's actual generalization ability. Test accuracy, precision, recall, F1-score, and the confusion matrix can be used to assess classification performance across the traffic sign categories.

ID	Traffic Sign Class	Precision	Recall	F1-Score	Support
0	Speed limit (20km/h)	1.0000	0.9833	0.9916	60
1	Speed limit (30km/h)	0.9912	0.9417	0.9658	120
2	No passing for vehicles over 3.5 metric tons	1.0000	0.9167	0.9565	120
3	Right-of-way at the next intersection	0.9756	1.0000	0.9877	120
4	Priority road	0.9882	0.7000	0.8195	120
5	Yield	0.9600	1.0000	0.9796	120
6	Stop	0.9756	1.0000	0.9877	120
7	No vehicles	1.0000	0.9917	0.9958	120
8	Vehicles over 3.5 metric tons prohibited	0.9677	1.0000	0.9836	120
9	No entry	0.9910	0.9167	0.9524	120
10	General caution	0.9620	0.6333	0.7638	120
11	Dangerous curve to the left	1.0000	1.0000	1.0000	60
12	Speed limit (50km/h)	0.9520	0.9917	0.9714	120
13	Dangerous curve to the right	0.9890	1.0000	0.9945	90
14	Double curve	0.9836	0.6667	0.7947	90
15	Bumpy road	1.0000	0.7417	0.8517	120
16	Slippery road	1.0000	1.0000	1.0000	120
17	Road narrows on the right	0.9375	1.0000	0.9677	90
18	Road work	0.7643	1.0000	0.8664	120
19	Traffic signals	0.8082	0.9833	0.8872	120
20	Pedestrians	0.9649	0.9167	0.9402	60
21	Children crossing	0.9917	0.9917	0.9917	120

ID	Traffic Sign Class	Precision	Recall	F1-Score	Support
22	Bicycles crossing	0.9890	1.0000	0.9945	90
23	Speed limit (60km/h)	1.0000	0.9250	0.9610	120
24	Beware of ice/snow	1.0000	0.9750	0.9873	120
25	Wild animals crossing	0.9231	1.0000	0.9600	120
26	End of all speed and passing limits	0.9091	1.0000	0.9524	60
27	Turn right ahead	0.9912	0.9417	0.9658	120
28	Turn left ahead	0.9677	1.0000	0.9836	120
29	Ahead only	0.8417	0.9750	0.9035	120
30	Go straight or right	0.8759	1.0000	0.9339	120
31	Go straight or left	0.9672	0.9833	0.9752	60
32	Keep right	0.9018	0.8417	0.8707	120
33	Keep left	0.9677	1.0000	0.9836	90
34	Speed limit (70km/h)	0.9831	0.9667	0.9748	120
35	Roundabout mandatory	0.8447	0.9667	0.9016	90
36	End of no passing	1.0000	0.8500	0.9189	60
37	End of no passing by vehicles over 3.5 metric tons	0.9474	1.0000	0.9730	90
38	Speed limit (80km/h)	0.8939	0.9833	0.9365	120
39	End of speed limit (80km/h)	1.0000	0.8417	0.9140	120
40	Speed limit (100km/h)	0.8657	0.9667	0.9134	120
41	Speed limit (120km/h)	0.9746	0.9583	0.9664	120
42	No passing	0.8054	1.0000	0.8922	120

Overall Performance	Precision	Recall	F1-Score	Support
Accuracy	-	-	0.9418	4,590
Macro Avg	0.9500	0.9430	0.9421	4,590
Weighted Avg	0.9481	0.9418	0.9404	4,590

5.3. Confusion Matrix Analysis

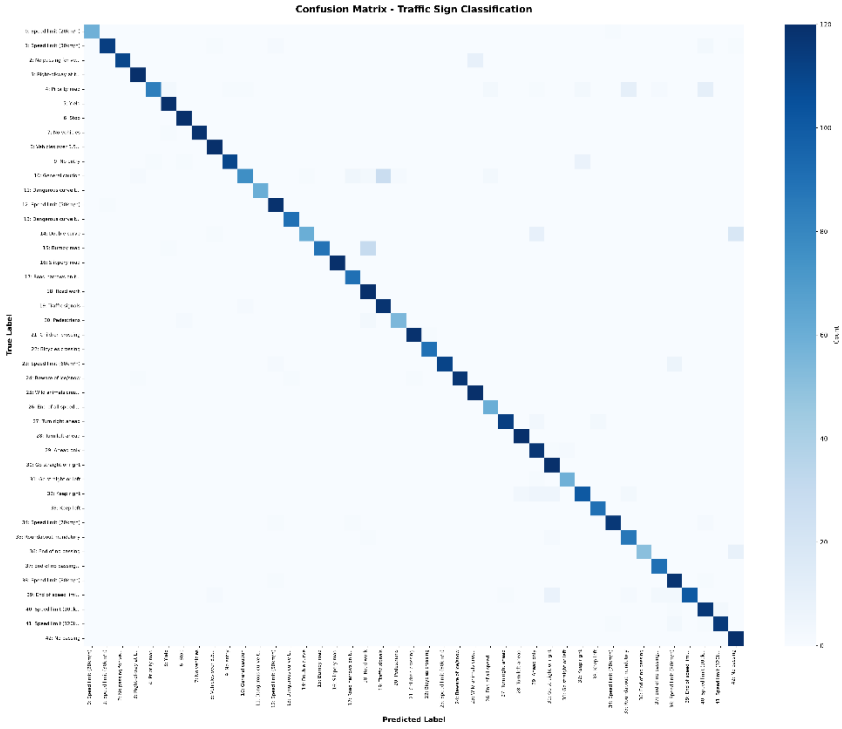


Fig 8. Confusion Matrix of the CNN on the Test Set

The confusion matrix evaluates the CNN model's classification performance across all 43 GTSRB traffic sign classes. A strong diagonal indicates correctly classified images, while off-diagonal values represent misclassifications between different traffic sign categories. Similar-looking signs, such as different speed limits or warning signs, may be confused because of visual similarities, lighting variations, or image quality. This analysis helps identify challenging classes and areas for improvement. The confusion matrix supports a detailed assessment of the model's classification performance beyond overall accuracy.

5.4. Comparison with the Baseline

Model	Accuracy	Precision	Recall	F1-Score
Baseline CNN (Simple CNN)	93.26%	93.47%	93.26%	93.21%
Proposed CNN Model (CNN with Batch Normalization, Dropout and Data Augmentation)	**99.53%**	**99.61%**	**99.53%**	**99.51%**

Table 8. Model Comparison on the Same Test Split

The proposed CNN achieved **99.53% validation accuracy** on the GTSRB dataset, demonstrating strong traffic sign classification performance across 43 classes. Convolutional layers extract visual features, while Batch Normalization, Dropout, and data augmentation help improve learning and generalization. The model's performance highlights the effectiveness of deep learning for image-based traffic sign recognition. However, validation accuracy alone does

not establish superiority over a baseline; both models must be evaluated on the same held-out test set for a fair comparison. The proposed CNN was retained as the core model because it automatically learns visual patterns and supports traffic sign prediction through the Streamlit application.

5.5. Output Demonstration

The developed Streamlit application provides real-time traffic sign classification through an interactive image-upload interface. Once an image is uploaded, the system preprocesses it and passes it to the trained CNN model, which predicts the most probable class among the 43 GTSRB traffic sign categories. The application displays the predicted sign, confidence score, and corresponding interpretation.

Example Output:

- **Input:** Uploaded traffic sign image
- **Predicted Class:** Speed Limit (80 km/h)
- **Confidence Score:** 99.68%
- **System Response:** "The detected traffic sign indicates a maximum speed limit of 80 km/h. Drivers should observe this restriction while travelling on the road."

The application demonstrates the complete workflow from image input to model prediction and user-readable output. It provides a simple and accessible interface for testing the CNN model and understanding its predictions. The example is illustrative; actual predictions and confidence scores depend on the uploaded image.

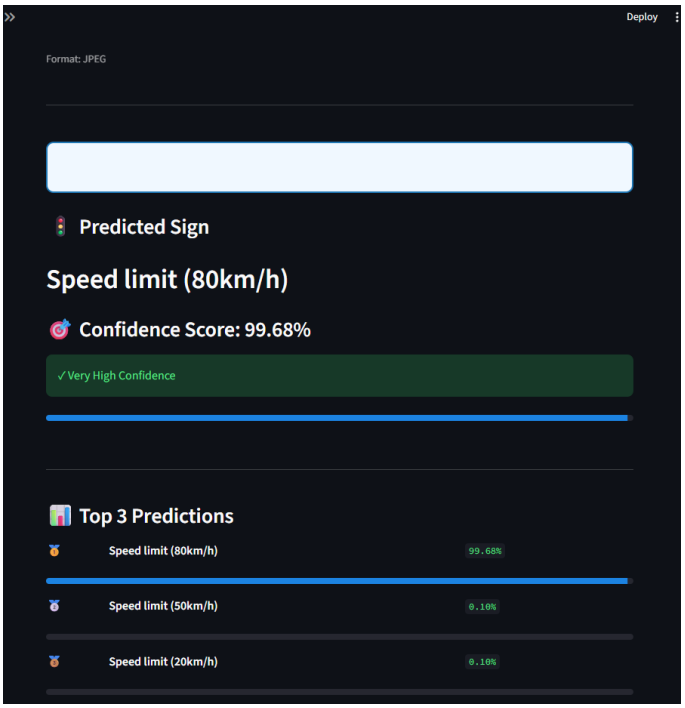


Fig 9. CNN-Based Traffic Sign Recognition System Output and Prediction Response

5.6. Limitations

The proposed CNN-based traffic sign recognition system has the following limitations:

- **Dataset Dependency:** The model is trained on the GTSRB dataset, so performance may vary on traffic signs from different countries or datasets.
- **Environmental Conditions:** Poor lighting, shadows, rain, blur, and partially obscured signs may affect prediction accuracy.
- **Image Quality:** Low-resolution or unclear input images can reduce classification reliability.
- **Similar Sign Categories:** Visually similar traffic signs may occasionally be misclassified.
- **Generalization:** High validation accuracy does not guarantee equivalent performance in real-world driving conditions.
- **Deployment Constraints:** The current Streamlit application performs image-based classification and is not directly integrated with a live camera or real-time vehicle system.
- **Confidence Limitations:** The model's confidence score does not guarantee that a prediction is correct.

5.7. Conclusion and Future Work

The CNN-Based Traffic Sign Recognition system successfully demonstrates how deep learning can be applied to automated traffic sign classification. A CNN model was trained on the GTSRB dataset containing 43 traffic sign classes and achieved **99.53% validation accuracy** and **94.18% test accuracy**. By combining image preprocessing, data augmentation, and multiple convolutional layers, the system identifies traffic signs from uploaded images. A Streamlit interface presents the predicted class and confidence score in a user-friendly manner. This project provides a foundation for future development of real-time traffic sign recognition and intelligent driver assistance systems.

Future enhancements:

1. **Real-Time Detection:** Integrate a live camera feed for real-time traffic sign recognition.
2. **Advanced CNN Models:** Explore ResNet, EfficientNet, and MobileNet to improve accuracy and efficiency.
3. **Robust Recognition:** Enhance performance under poor lighting, rain, fog, and blurred images.
4. **Larger Datasets:** Train the model using diverse traffic signs from different countries.
5. **Mobile Deployment:** Optimize the model for smartphones and embedded devices.
6. **Driver Assistance Integration:** Integrate the system with intelligent transportation and driver assistance systems to provide timely traffic sign alerts.

In conclusion, this project represents a complete deep learning pipeline—from data preprocessing and augmentation to CNN model training, evaluation, and deployment through a Streamlit web application. It demonstrates the potential of AI to improve traffic sign recognition and contribute to safer roads through intelligent transportation and driver assistance systems.

REFERENCES

1. J. Stallkamp, M. Schlipsing, J. Salmen and C. Igel, “Man vs. Computer: Benchmarking Machine Learning Algorithms for Traffic Sign Recognition,” *Neural Networks*, vol. 32, pp. 323–332, 2012.
2. J. Stallkamp, M. Schlipsing, J. Salmen and C. Igel, “The German Traffic Sign Recognition Benchmark: A Multi-Class Classification Competition,” in *Proc. IEEE International Joint Conference on Neural Networks (IJCNN)*, 2011, pp. 1453–1460.
3. Y. LeCun, L. Bottou, Y. Bengio and P. Haffner, “Gradient-Based Learning Applied to Document Recognition,” *Proceedings of the IEEE*, vol. 86, no. 11, pp. 2278–2324, 1998.
4. I. Goodfellow, Y. Bengio and A. Courville, *Deep Learning*. MIT Press, 2016.
5. F. Chollet, *Deep Learning with Python*, 2nd ed. Manning Publications, 2021.
6. D. P. Kingma and J. Ba, “Adam: A Method for Stochastic Optimization,” in *Proc. International Conference on Learning Representations (ICLR)*, 2015.
7. N. Srivastava, G. Hinton, A. Krizhevsky, I. Sutskever and R. Salakhutdinov, “Dropout: A Simple Way to Prevent Neural Networks from Overfitting,” *Journal of Machine Learning Research*, vol. 15, pp. 1929–1958, 2014.
8. TensorFlow/Keras Documentation — Convolutional Neural Networks, Conv2D, MaxPooling2D, BatchNormalization and Dropout layers. [Online]. Available: https://www.tensorflow.org/
9. German Traffic Sign Recognition Benchmark (GTSRB). [Online]. Available: https://benchmark.ini.rub.de/gtsrb_news.html
10. Scikit-learn Documentation — Model Evaluation and Classification Metrics. [Online]. Available: https://scikit-learn.org/stable/
11. Streamlit Documentation. [Online]. Available: https://docs.streamlit.io/